Querying, Visualizing, and Versioning: A Common Table Expression Analysis of the Sakila Rental Database

Nikita Mishra

College of Professional Studies, Northeastern University

ALY 6600: Data Warehousing & Querying

Professor Shahram Sattar

November 7, 2026

Abstract

This report documents Lab Assignment 8 in three parts (Write, Explain, and Extend). It applied common table expressions (CTEs), a business intelligence (BI) connection, and Git version control to the Sakila DVD rental database hosted in Snowflake. A single CTE identified 285 customers whose lifetime spend exceeded the average customer total, which sat just below $112.70; these customers spent a combined $38,024.18. A chain of three CTEs ranked the two stores by their above-average customers: Lethbridge ranked first with 160 customers and $21,285.61 in revenue (56.0%), ahead of Woodridge with 125 customers and $16,738.57 (44.0%). Revenue per above-average customer was almost identical at the two stores ($133.04 and $133.91), so Lethbridge's lead comes from having more high-value customers rather than from higher spending per customer. A Power BI DirectQuery dashboard showed that July and August 2005 produced 78.4% of this revenue. A recency classification measured against the dataset's final rental date (February 14, 2006) found that the 20 highest-spending At Risk customers had last rented 175 to 177 days earlier and had a combined lifetime spend of $3,488.79. The findings highlight both the value of the store's top customers and the limits of recency measures on historical sample data.

Keywords: common table expressions, Snowflake, Power BI, DirectQuery, Git, customer recency

Querying, Visualizing, and Versioning: A Common Table Expression Analysis of the Sakila Rental Database

Introduction

Analytics engineers rarely answer a business question with a single, flat query. They break the problem into named steps, put the result in front of decision makers through a dashboard, and keep the code under version control so that a team can trust and reproduce it. Lab Assignment 8 practised this query, visualize, and version workflow on the Sakila sample database, which models the customers, rentals, payments, and stores of a fictional DVD rental business (Oracle, n.d.).

The report follows the structure of the assignment brief. Part 1 (Write) covers Task 1, a single CTE that finds above-average customers; Task 2, chained CTEs that rank the stores; Task 3, a Power BI connection and visualization; and Task 4, a Git commit and push. Part 2 (Explain) answers the four comprehension questions. Part 3 (Extend) presents a customer recency classification. Each task describes the approach, reports the results with the figures returned by the queries, and interprets what they mean for the business.

Data and Tools

All queries ran in Snowflake against the normalized Sakila schema (SAKILA.NORM) using the ALY6600_WH warehouse and the ALY6600_STUDENT role. In this schema the PAYMENT table does not store a customer identifier, so every spending calculation joins PAYMENT to RENTAL to attribute each payment to a customer. Customers are assigned to stores through their home store (CUSTOMER.STORE_ID). Power BI Desktop was used for the visualization, and Git was used for version control. The complete SQL for every task is submitted in the accompanying .sql file.

Part 1: Write

1.1 Task 1: Single CTE

Approach

One CTE, customer_spend, totals each customer's payments. The outer query keeps customers whose total is greater than the average of those totals and orders them by total spend, highest first. Averaging the per-customer totals, rather than individual payment rows, keeps the comparison at the customer grain (Snowflake, n.d.-b).

SQL Query

```
WITH customer_spend AS (
    -- One row per customer: lifetime total of their payments
    SELECT
        r.customer_id,
        SUM(p.amount) AS total_spend
    FROM payment AS p
    JOIN rental  AS r
        ON r.rental_id = p.rental_id
    GROUP BY r.customer_id
)
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    cs.total_spend
FROM customer_spend AS cs
JOIN customer AS c
    ON c.customer_id = cs.customer_id
WHERE cs.total_spend > (SELECT AVG(total_spend) FROM customer_spend)
ORDER BY cs.total_spend DESC;
```

Results

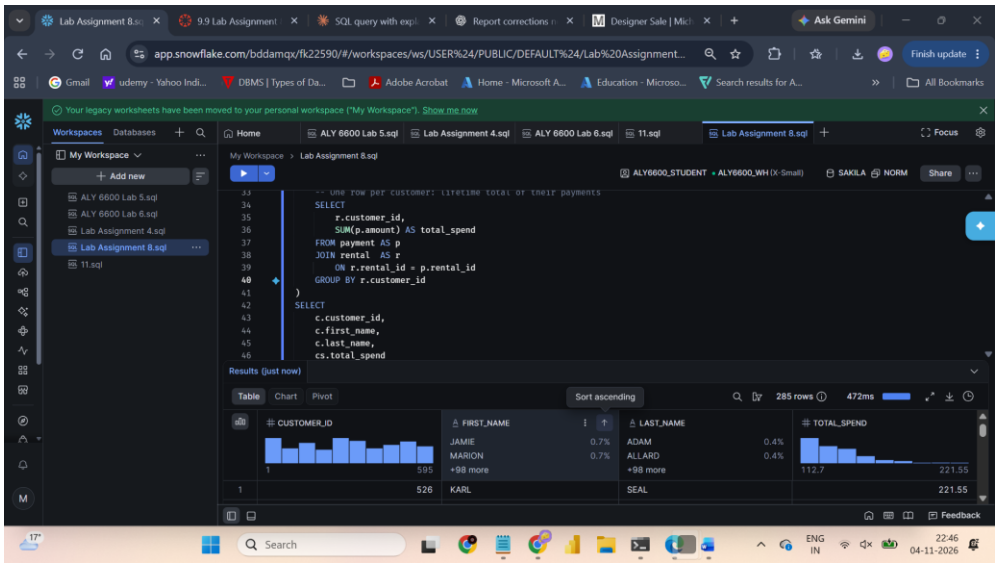
The query returned 285 customers (Figure 1). The lowest qualifying customer spent $112.70, which places the average customer total just below that amount. Within this group, spending ranged from $112.70 to $221.55 (Karl Seal), with a median of $129.70 and a mean of $133.42. Together these customers spent $38,024.18.

Interpretation

The spread among above-average customers is moderate: most sit within about $20 of $130, and only a few, such as Karl Seal and Eleanor Hunt ($216.54), stand well apart. These 285 customers form the high-value segment used in every later task.

Figure 1

Task 1 Query Results in Snowflake: Customers With Above-Average Total Spend



Note. Rows are ordered by total spend in descending order.

1.2 Task 2: Chained CTEs

Approach

Three chained CTEs build the answer in steps: customer_spend totals spend per customer, above_avg_customers keeps those above the average, and above_avg_customers_by_store attaches each customer's home store and city. A final SELECT counts the customers and sums their revenue by store, ranks the stores with RANK (customer count first, revenue as the tie-breaker), and applies LIMIT 5.

SQL Query

```
WITH customer_spend AS (
    -- Step 1: total spend per customer
    SELECT
```

```
        r.customer_id,
        SUM(p.amount) AS total_spend
    FROM payment AS p
    JOIN rental  AS r
        ON r.rental_id = p.rental_id
    GROUP BY r.customer_id
),

above_avg_customers AS (
    -- Step 2: keep only customers above the average customer total
    SELECT
        customer_id,
        total_spend
    FROM customer_spend
    WHERE total_spend > (SELECT AVG(total_spend) FROM customer_spend)
),

above_avg_customers_by_store AS (
    -- Step 3: attach each customer's home store and its city label
    SELECT
        a.customer_id,
        a.total_spend,
        s.store_id,
        ci.city AS store_city
    FROM above_avg_customers AS a
    JOIN customer AS c
        ON c.customer_id = a.customer_id
    JOIN store    AS s
        ON s.store_id = c.store_id
    JOIN address  AS ad
        ON ad.address_id = s.address_id
    JOIN city     AS ci
        ON ci.city_id = ad.city_id
)

-- Final step: aggregate by store, rank, and keep the top five
SELECT
    RANK() OVER (
        ORDER BY COUNT(customer_id) DESC, SUM(total_spend) DESC
    )                          AS store_rank,
    store_id,
    store_city,
    COUNT(customer_id)         AS above_avg_customer_count,
    SUM(total_spend)           AS revenue_from_above_avg_customers
FROM above_avg_customers_by_store
GROUP BY store_id, store_city
ORDER BY store_rank
LIMIT 5;
```

Results

Because the Sakila database contains only two stores, the top-five limit returned two rows (Table 1; Figure 2). Lethbridge ranked first with 160 above-average customers and $21,285.61 in revenue, or 56.0% of the $38,024.18 total. Woodridge ranked second with 125 customers and $16,738.57 (44.0%).

Table 1

Above-Average Customers and Revenue by Store

Rank	Store	Above-average customers	Revenue ($)	Share of revenue	Revenue per customer ($)
1	Lethbridge	160	21,285.61	56.0%	133.04
2	Woodridge	125	16,738.57	44.0%	133.91
	Total	285	38,024.18	100.0%	133.42

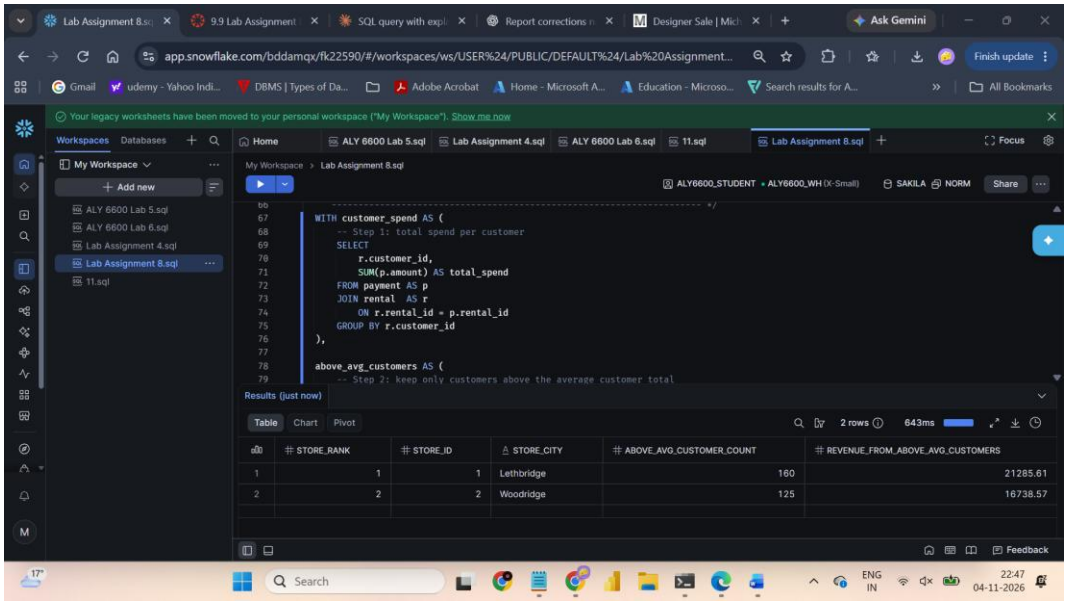
Note. Stores are ranked by number of above-average customers. Revenue is the lifetime spend of each store's above-average customers, attributed to the customer's home store.

Interpretation

Each above-average customer spent almost the same amount at either store: $133.04 at Lethbridge and $133.91 at Woodridge. Lethbridge's lead therefore comes from having 35 more high-value customers, not from those customers spending more per head. Growing Woodridge's revenue is mainly a question of moving more of its customers above the average, for example through loyalty offers, rather than raising prices.

Figure 2

Task 2 Query Results in Snowflake: Stores Ranked by Above-Average Customers



Note. The query uses three chained CTEs followed by a final SELECT with RANK and LIMIT 5.

1.3 Task 3: BI Tool Connection

Approach

Power BI Desktop was connected to Snowflake in DirectQuery mode, with a variant of the Task 2 query entered as a native SQL statement. The variant breaks revenue from above-average customers out by store and payment month, which gives the dashboard a date to filter on. The visualization is a clustered column chart with a title, subtitle, and labelled axes, plus two filters: a Payment Month date-range slicer and a Store City slicer (Figure 3). In Power BI, each table name was prefixed with SAKILA.NORM. because the native SQL statement runs without the USE SCHEMA setting.

SQL Query

```
WITH customer_spend AS (
    SELECT
        r.customer_id,
        SUM(p.amount) AS total_spend
    FROM SAKILA.NORM.PAYMENT AS p
    JOIN SAKILA.NORM.RENTAL  AS r
        ON r.rental_id = p.rental_id
    GROUP BY r.customer_id
),

above_avg_customers AS (
    SELECT customer_id
    FROM customer_spend
    WHERE total_spend > (SELECT AVG(total_spend) FROM customer_spend)
)

SELECT
    c.store_id,
    ci.city                                 AS store_city,
    DATE_TRUNC('month', p.payment_date)   AS payment_month,
    COUNT(DISTINCT r.customer_id)         AS above_avg_customer_count,
    SUM(p.amount)                           AS revenue_from_above_avg_customers
FROM SAKILA.NORM.PAYMENT AS p
JOIN SAKILA.NORM.RENTAL   AS r   ON r.rental_id    = p.rental_id
JOIN above_avg_customers AS a ON a.customer_id = r.customer_id
JOIN SAKILA.NORM.CUSTOMER AS c   ON c.customer_id  = r.customer_id
JOIN SAKILA.NORM.STORE    AS s   ON s.store_id     = c.store_id
JOIN SAKILA.NORM.ADDRESS  AS ad  ON ad.address_id  = s.address_id
JOIN SAKILA.NORM.CITY     AS ci  ON ci.city_id     = ad.city_id
GROUP BY c.store_id, ci.city, DATE_TRUNC('month', p.payment_date)
ORDER BY payment_month, c.store_id;
```

Results

Table 2 lists the values behind the chart. Revenue rose from $2,625.83 in May 2005 to a peak of $16,128.25 in July 2005, then eased to $13,667.28 in August. July and August together produced $29,795.53, or 78.4% of the total. Lethbridge earned more than Woodridge in every month.

Table 2

Monthly Revenue From Above-Average Customers by Store

Payment month	Lethbridge ($)	Woodridge ($)	Total ($)	Share of total
May 2005	1,501.52	1,124.31	2,625.83	6.9%
June 2005	2,861.33	2,487.38	5,348.71	14.1%
July 2005	9,277.21	6,851.04	16,128.25	42.4%
August 2005	7,506.05	6,161.23	13,667.28	35.9%
February 2006	139.50	114.61	254.11	0.7%
Total	21,285.61	16,738.57	38,024.18	100.0%

Note. Values are from the Task 3 query used in Power BI. No payments were recorded for September 2005 through January 2006.

Interpretation

Revenue from high-value customers is concentrated in the summer of 2005. The query returned no payments for September 2005 through January 2006, and February 2006 contained only $254.11. This gap is a feature of the sample data rather than a real collapse in business, but any trend or recency measure spanning these months must be read with it in mind.

Figure 3

Power BI Dashboard: Monthly Revenue From Above-Average Customers by Store



Note. Power BI Desktop connected to Snowflake in DirectQuery mode.

1.4 Task 4: Git Basics

The Lab 8 SQL file was uploaded to the GitHub repository to maintain and share the project files. The repository contains the SQL assignment file, lab8_nikita_mishra.sql, and a README.md file describing the project, its objectives, SQL tasks, and key findings.

GitHub Repository: https://github.com/NikitaMishra-Analytics/ALY6600-Lab8

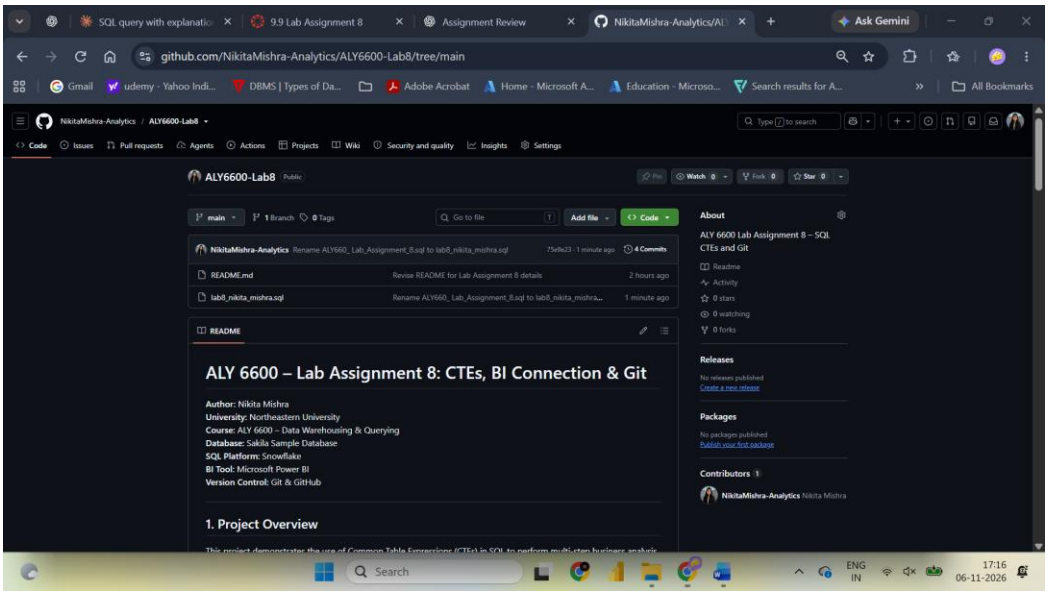
The files were uploaded and committed to the repository. The commit history provides a record of the changes made to the project. Figure 4 shows the repository files and commit history.

Results

The repository was successfully created and made publicly accessible. Both the SQL file and README are available in the repository for review.

Figure 4

Git Commit History Showing the Lab 8 Commit



Note. The commit adds lab8_nikita_mishra.sql to the repository.

Part 2: Explain

2.1 Question 1: CTE Versus Subquery Readability

Task 1 Rewritten With a Subquery

```
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    cs.total_spend
FROM (
        SELECT r.customer_id, SUM(p.amount) AS total_spend
        FROM payment AS p
        JOIN rental  AS r ON r.rental_id = p.rental_id
        GROUP BY r.customer_id
     ) AS cs
JOIN customer AS c
    ON c.customer_id = cs.customer_id
WHERE cs.total_spend > (
        SELECT AVG(t.total_spend)
        FROM (
                SELECT r.customer_id, SUM(p.amount) AS total_spend
                FROM payment AS p
                JOIN rental  AS r ON r.rental_id = p.rental_id
                GROUP BY r.customer_id
             ) AS t
      )
ORDER BY cs.total_spend DESC;
```

Rewriting Task 1 with a subquery in the FROM clause returns the same 285 customers, but the per-customer aggregation has to be written twice: once as the derived table and again inside the WHERE clause, because a derived table in FROM cannot be referenced from the WHERE-clause subquery. The CTE version is more readable because the aggregation is written once, named customer_spend, and reused, so the reader can follow it top to bottom instead of inside out. With one step used once, the two styles are about equal. The CTE advantage becomes significant as soon as a step is used more than once, as in Task 1, and decisive at two or more levels of nesting. Task 2 has three dependent steps, which as subqueries would be nested three levels deep with the spending logic repeated.

2.2 Question 2: CTE Reference Order

No. In Task 2, customer_spend cannot reference above_avg_customers_by_store. CTEs are resolved sequentially from top to bottom, so each CTE can reference only base tables and CTEs defined above it in the same WITH clause (Snowflake, n.d.-b). The reference would also be circular, because the third CTE is built from the second, which is built from the first. The exception is a recursive CTE (WITH RECURSIVE), which references itself to walk a hierarchy; that is self-reference with an anchor, not a forward reference, and Task 2 does not need it.

2.3 Question 3: Dashboard Filters, Live Connection, and Extract

The answer depends on the connection pattern. In a live connection (DirectQuery in Power BI), the BI tool does not filter locally: it wraps the query, adds a WHERE clause for the filter, and sends the new SQL to the warehouse, which re-executes it (Microsoft, n.d.). Data is always current, but each click uses warehouse compute. In an extract (Import in Power BI), the query runs once at refresh time, the result is stored in the BI tool, and filters are applied locally, which is fast but only as fresh as the last refresh. My setup uses DirectQuery, so each change to the Payment Month or Store City slicer sends a

new filtered query to Snowflake. Because the Task 2 query ends in LIMIT 5, a filter wrapped around it would filter the already-ranked stores rather than re-rank them, which is why the Task 3 variant returns unranked store-month rows.

2.4 Question 4: Why dbt Requires Git

In dbt, every model is a SQL file, and the code in the repository is the source of truth for what gets built in the warehouse (dbt Labs, n.d.). Git provides the history, review, branching, and rollback that make this safe (Chacon & Straub, 2014). Without version control, if a teammate and I both edited lab8_nikita_mishra.sql, for example one of us changing how customer_spend is calculated and the other renaming a CTE, whoever saved last would silently overwrite the other's work, and no one could tell which version produced the published results. With Git, as in Task 4, the second push is rejected until that person pulls the latest version, and any edits to the same lines are flagged as a merge conflict that must be resolved deliberately.

Part 3: Extend

3.1 Customer Recency Classification

Approach

Separate CTEs calculate a reference date, each customer's most recent rental and days since that rental, each customer's lifetime spend, and the CASE WHEN classification. Customers who rented within 90 days are Active, those at 91 to 180 days are At Risk, and those beyond 180 days are Lapsed. Recency is measured in whole days with DATEDIFF (Snowflake, n.d.-a) against the latest rental date in the data, February 14, 2006, rather than the current date, because the Sakila records end in 2006. Whole-day counts mean the bands meet at 90/91 and 180/181 days without gaps or overlaps. The final SELECT keeps At Risk customers, orders them by lifetime spend, and applies LIMIT 20.

SQL Query

```
WITH reference_date AS (
    -- Single-row "as of" date: the last rental recorded in the data
    SELECT MAX(rental_date) AS as_of_date
    FROM rental
),

customer_recency AS (
    -- Recency calculation: most recent rental and days since it
    SELECT
        r.customer_id,
        MAX(r.rental_date)                                    AS
most_recent_rental_date,
        DATEDIFF('day', MAX(r.rental_date), rd.as_of_date)   AS
days_since_last_rental
    FROM rental AS r
    CROSS JOIN reference_date AS rd
    GROUP BY r.customer_id, rd.as_of_date
),

customer_lifetime_spend AS (
    -- One row per customer: lifetime total of payments
    SELECT
        r.customer_id,
        SUM(p.amount) AS lifetime_spend
    FROM payment AS p
    JOIN rental  AS r
        ON r.rental_id = p.rental_id
    GROUP BY r.customer_id
),

customer_recency_band AS (
    -- Classification using CASE WHEN on whole-day recency
    SELECT
        cr.customer_id,
        cr.most_recent_rental_date,
        cr.days_since_last_rental,
        CASE
            WHEN cr.days_since_last_rental <= 90  THEN 'Active'
            WHEN cr.days_since_last_rental <= 180 THEN 'At Risk'
            ELSE 'Lapsed'
        END AS recency_band
    FROM customer_recency AS cr
)

SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    rb.most_recent_rental_date,
    rb.days_since_last_rental,
    rb.recency_band,
    ls.lifetime_spend
FROM customer_recency_band AS rb
JOIN customer_lifetime_spend AS ls
    ON ls.customer_id = rb.customer_id
JOIN customer AS c
    ON c.customer_id = rb.customer_id
```

```
WHERE rb.recency_band = 'At Risk'
ORDER BY ls.lifetime_spend DESC, c.customer_id
LIMIT 20;
```

Results

Table 3 and Figure 5 show the 20 At Risk customers with the highest lifetime spend. All of them last rented between August 21 and August 23, 2005: 18 customers 175 days before the reference date, Mike Way 176 days before, and Gordon Allard 177 days before. Their combined lifetime spend is $3,488.79, led by Karl Seal ($221.55) and Eleanor Hunt ($216.54).

Table 3

Top 20 At Risk Customers by Lifetime Spend

Rank	Customer	Most recent rental	Days since	Lifetime spend ($)
1	Karl Seal	Aug 23, 2005	175	221.55
2	Eleanor Hunt	Aug 23, 2005	175	216.54
3	Clara Shaw	Aug 23, 2005	175	195.58
4	Rhonda Kennedy	Aug 23, 2005	175	194.61
5	Tommy Collazo	Aug 23, 2005	175	186.62
6	Wesley Bull	Aug 23, 2005	175	177.60
7	Tim Cary	Aug 23, 2005	175	175.61
8	June Carroll	Aug 23, 2005	175	173.63
9	Diane Collins	Aug 23, 2005	175	169.65
10	Lena Jensen	Aug 23, 2005	175	168.68
11	Arnold Havens	Aug 23, 2005	175	167.67
12	Mike Way	Aug 22, 2005	176	166.65
13	Tonya Chapman	Aug 23, 2005	175	161.68
14	Gordon Allard	Aug 21, 2005	177	160.68
15	Brittany Riley	Aug 23, 2005	175	159.72
16	Warren Sherrod	Aug 23, 2005	175	159.67
17	Mae Fletcher	Aug 23, 2005	175	158.69
18	Steve Mackenzie	Aug 23, 2005	175	158.66
19	Priscilla Lowe	Aug 23, 2005	175	157.65
20	Harry Arce	Aug 23, 2005	175	157.65

Rank	Customer	Most recent rental	Days since	Lifetime spend ($)
	Total			3,488.79

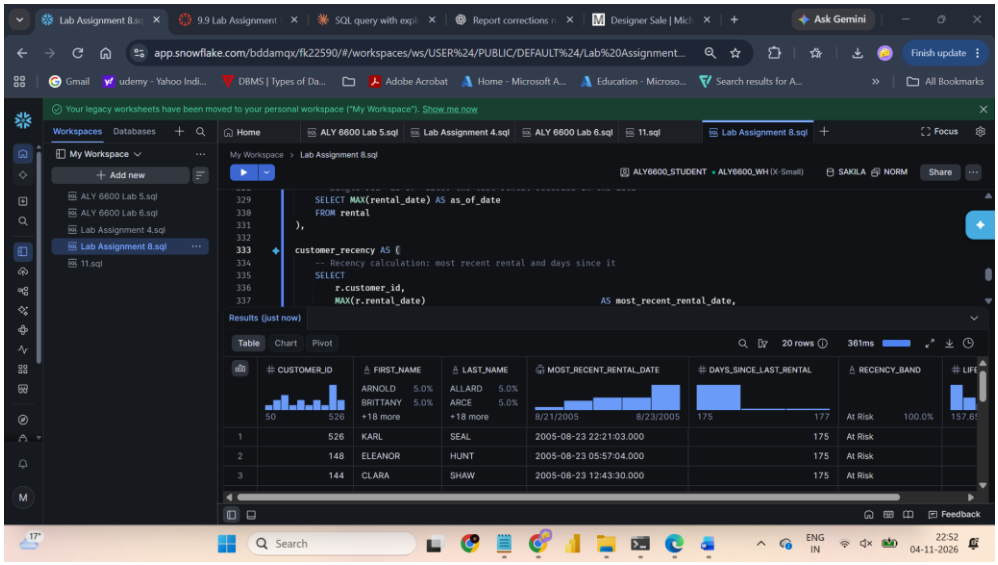
Note. Days since last rental are measured against February 14, 2006, the latest rental date in the dataset. At Risk = 91 to 180 days.

Interpretation

Every one of these customers also appears in the Task 1 list of above-average spenders, so the business's most valuable customers are close to the 180-day line that would move them to Lapsed. They would be the first target for a win-back campaign. The result must still be read with care: the clustering at 175 days follows from the gap in the data, because rental activity effectively stops in late August 2005 apart from a small batch in February 2006. The classification is defensible as recency relative to the end of the dataset, but it describes the sample rather than customer behaviour today.

Figure 5

Part 3 Query Results in Snowflake: Top 20 At Risk Customers



Note. The query uses CTEs for the reference date, recency, lifetime spend, and CASE WHEN classification.

Conclusion

This lab showed how CTEs turn a multi-step business question into a readable sequence of named steps, and how the same logic can feed a live dashboard and be kept under version control. The analysis found 285 above-average customers who together spent $38,024.18. Lethbridge leads Woodridge because it has more of these customers (160 against 125), while spending per customer is nearly identical at about $133 at both stores. Revenue from these customers was concentrated in July and August 2005, which together account for 78.4% of the total.

The recency analysis identified 20 high-value customers, worth $3,488.79 combined, who are At Risk and close to lapsing, which makes them a clear priority for retention. Two limitations apply. First, recency was measured against the last rental in a historical sample, and the missing months from September 2005 to January 2006 push many customers into the same band. Second, revenue was attributed to each customer's home store rather than to the store where each rental took place, so a store-of-rental view could shift the totals slightly. With live data and a current reference date, the same CTE pipeline could support an ongoing customer-retention dashboard.

References

Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book/en/v2

dbt Labs. (n.d.). *What is dbt?* dbt Developer Hub. https://docs.getdbt.com/docs/introduction

Microsoft. (n.d.). *DirectQuery in Power BI*. Microsoft Learn. https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-directquery-about

Oracle. (n.d.). *Sakila sample database*. MySQL Documentation. https://dev.mysql.com/doc/sakila/en/

Snowflake. (n.d.-a). *DATEDIFF*. Snowflake Documentation. https://docs.snowflake.com/en/sql-reference/functions/datediff

Snowflake. (n.d.-b). *WITH*. Snowflake Documentation. https://docs.snowflake.com/en/sql-reference/constructs/with